

Algoritmizálás C++-ban: összetett feladatok

Összefoglaló feladatlap

Hogyan dolgozz ezzel a feladatlappal?

Ebben a feladatlapban nincs új tananyag. Azt gyakorlod, hogyan rakd össze egy nagyobb feladat megoldását abból, amit a vektorokról, a sztringekről, a függvényekről és a structokról tanultál.

- **Egy fejezet egy nagy feladat.** A részfeladatok egymásra épülnek, ezért sorban haladj. Ugyanazt a programot bővítsd kérdésről kérdésre.
- **Ellenőrzés.** Minden részfeladatnál ott van, mit kell kapnod. Csak akkor lépj tovább, ha egyezik.
- **Segítség.** A feladatlap végén minden részfeladathoz találsz egy rövid segítséget. Előbb próbáld meg nélküle: csak akkor lapozz oda, ha legalább öt percig próbálkoztál.

A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb. Nem kell minden fejezettel végezned. Az 1-3. fejezet mindenkinek szól; utána a 4-9. fejezetek közül választhatsz is.

1. Hogyan állj neki egy összetett feladatnak?

Egy nagy feladat ijesztő lehet, ha egyszerre akarod megoldani. Bontsd lépésekre:

1. **Olvasd végig az egész feladatot.** Mi a bemenet? Mit kell kiírni? Húzd alá a kérdésekben a kulcsszavakat.
2. **Tervezd meg az adatokat.** Mit kell eltárolni? Ha egy sor több összetartozó adatot tartalmaz, abból struct lesz. Sok sorból structok vektora.
3. **Olvasd be az adatokat, és ellenőrizd.** Írd ki, hány elemet olvastál be, és az első elem mezőit. Amíg ez nem jó, ne menj tovább.
4. **Kérdésenként haladj.** Döntsd el, melyik minta kell hozzá (2. fejezet), írd meg, futtasd, és hasonlítsd össze az ellenőrző értékkel.
5. **Csak ezután jöhet a következő kérdés.** Amit már megírtál és működik, ahhoz ne nyúlj.

Ha nem azt kapod, amit kell

- Írasd ki a közbülső értékeket: a ciklusváltozót, az összeget, az éppen vizsgált elemet. Így látod, hol romlik el.
- Próbáld ki kevesebb adattal, amit fejben is végig tudsz követni.
- Nézd meg a ciklus határait: az első és az utolsó elem is sorra kerül? Nem lépsz túl a vektor végén?
- Átlagnál: nem egész osztás történt?
- Szövegből lett számnál: megvolt a `std::stoi`?

2. A minták egy helyen

Szinte minden kérdés visszavezethető néhány jól ismert mintára. A kérdés megfogalmazása elárulja, melyik kell. A példákban a **v** dolgozatok vektora, minden elemének van **nev** és **pont** mezője.

Összegzés

Erről ismered fel: „mennyi összesen”, „mennyi az átlag”.

```
int osszeg = 0;
for (int i = 0; i < v.size(); i++) {
    osszeg = osszeg + v[i].pont;
}
```

Megszámlálás

Erről ismered fel: „hány”, „hányan”, „hányszor”.

```
int db = 0;
for (int i = 0; i < v.size(); i++) {
    if (v[i].pont >= 40) {
        db++;
    }
}
```

Maximum- és minimumkiválasztás

Erről ismered fel: „legnagyobb”, „legjobb”, „legkorábbi”, „melyik a leg...”. Az indexet jegyezd meg, mert így a nyertes minden adatát eléred.

```
int maxIndex = 0;
for (int i = 1; i < v.size(); i++) {
    if (v[i].pont > v[maxIndex].pont) {
        maxIndex = i;
    }
}
```

Eldöntés

Erről ismered fel: „van-e”, „volt-e”, „igaz-e, hogy minden”.

```
bool van = false;
for (int i = 0; i < v.size(); i++) {
    if (v[i].pont == 50) {
        van = true;
    }
}
```

Keresés

Erről ismered fel: „hol van”, „melyik az első”, „add meg az adott nevű”. Ha nincs találat, a **hely** értéke -1 marad.

```
int hely = -1;
for (int i = 0; i < v.size(); i++) {
    if (v[i].nev == keresett && hely == -1) {
        hely = i;
    }
}
```

Kiválogatás

Erről ismered fel: „sorold fel azokat”, „gyűjtsd ki”.

```
std::vector<Dolgozat> jok;
for (int i = 0; i < v.size(); i++) {
    if (v[i].pont >= 40) {
        jok.push_back(v[i]);
    }
}
```

Rendezés

Erről ismered fel: „sorrendben”, „rangsor”, „növekvő”, „csökkenő”. Összehasonlító függvény kell hozzá (struct jegyzet, 7. fejezet).

2.1. feladat ★☆☆ - Melyik minta kell?

Ezt a feladatot papíron oldd meg, program nélkül. Egy dolgozat eredményeiről szólnak a kérdések. Írd mindegyik mellé, melyik mintával válaszolnád meg.

1. Hány tanuló kapott ötöst?
2. Mennyi a dolgozatok átlagpontszáma?
3. Ki írta a legjobb dolgozatot?
4. Volt-e hibátlan dolgozat?
5. Hány pontot ért el Szabo Dani?
6. Sorold fel azokat, akik megbuktak!
7. Igaz-e, hogy mindenki elérte a 10 pontot?
8. Írd ki a tanulókat pontszám szerint csökkenő sorrendben!
9. Hányadik helyen végzett Varga Emma?

Ellenőrzés: a megoldást a feladatlap végén, a segítségek között találod. A 9. kérdés a trükkös: gondold végig, mit kell megszámolni hozzá.

3. Kidolgozott feladat: dolgozat

Ezt a feladatot együtt oldjuk meg, lépésről lépésre. A többi fejezetben ugyanezt az utat járod végig önállóan.

A feladat. Egy 50 pontos dolgozat eredményei soronként érkeznek, `nev;pontszam` formában:

```
Kiss Anna;42
Nagy Bence;28
Toth Csilla;47
Szabo Dani;19
Varga Emma;35
Kovacs Feri;42
```

A kérdések: (1) Hány dolgozat van? (2) Mennyi az átlagpontszám? (3) Ki írta a legjobb dolgozatot? (4) Hányan nem érték el a 20 pontot? (5) Volt-e 50 pontos dolgozat?

Az adatok megtervezése. Egy sor két összetartozó adat: név és pontszám. Ebből `Dolgozat` nevű struct lesz, a sok sorból pedig `std::vector<Dolgozat>`.

A beolvasás. Egy `feldolgoz` függvény alakít egy sort structtá. Hogy ne kelljen minden futtatásnál begépelned az adatokat, a sorokat most beírjuk a programba, egy sztringeket tároló vektorba. A feldolgozó részen akkor sem kellene változtatni, ha a sorok máshonnan érkeznének.

A kérdések. Mindegyik fölé megjegyzésben odaírtuk, melyik minta kell hozzá.

Próbáld ki

```
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

struct Dolgozat {
    std::string nev;
    int pont;
};

Dolgozat feldolgoz(std::string sor) {
    std::stringstream ss(sor);
    std::string pontSzoveg;

    Dolgozat d;
    std::getline(ss, d.nev, ';');
    std::getline(ss, pontSzoveg, ';');
    d.pont = std::stoi(pontSzoveg);
    return d;
}

int main() {
    std::vector<std::string> sorok = {
        "Kiss Anna;42",
        "Nagy Bence;28",
        "Toth Csilla;47",
        "Szabo Dani;19",
        "Varga Emma;35",
        "Kovacs Feri;42"
    };

    std::vector<Dolgozat> v;
    for (int i = 0; i < sorok.size(); i++) {
        v.push_back(feldolgoz(sorok[i]));
    }

    // 1. kerdes: darabszam
    std::cout << "1. Dolgozatok szama: " << v.size() << std::endl;

    // 2. kerdes: osszegzes
    double osszeg = 0;
    for (int i = 0; i < v.size(); i++) {
        osszeg = osszeg + v[i].pont;
    }
    std::cout << "2. Atlag: " << osszeg / v.size() << std::endl;

    // 3. kerdes: maximumkivalasztas
    int maxIndex = 0;
    for (int i = 1; i < v.size(); i++) {
        if (v[i].pont > v[maxIndex].pont) {
            maxIndex = i;
        }
    }
    std::cout << "3. Legjobb: " << v[maxIndex].nev << std::endl;

    // 4. kerdes: megszamlalas
    int db = 0;
    for (int i = 0; i < v.size(); i++) {
        if (v[i].pont < 20) {
            db++;
        }
    }
    std::cout << "4. 20 pont alatt: " << db << std::endl;
```

```

// 5. kérdés: eldöntes
bool van = false;
for (int i = 0; i < v.size(); i++) {
    if (v[i].pont == 50) {
        van = true;
    }
}
if (van) {
    std::cout << "5. Volt 50 pontos." << std::endl;
} else {
    std::cout << "5. Nem volt 50 pontos." << std::endl;
}
return 0;
}

```

Gépeled be, és futtasd. Ezt kell kapnod:

```

1. Dolgozatok száma: 6
2. Átlag: 35.5
3. Legjobb: Toth Csilla
4. 20 pont alatt: 1
5. Nem volt 50 pontos.

```

Figyeld meg a kiírást: minden válasz elé odaírtuk a kérdés sorszámát. Az érettségien is így kell.

Feladatok

Most te folytatod: bővítsd a fenti programot újabb kérdésekkel.

3.1. feladat ★☆☆ - Leggyengébb dolgozat

6. kérdés: ki írta a leggyengébb dolgozatot, és hány pontos lett?

Ellenőrzés: Szabo Dani, 19 pont.

3.2. feladat ★☆☆ - Érdemjegy

Írj a `Dolgozat` structba `int jegy()` tagfüggvényt. A ponthatárok: 0–19 pont elégtelen (1), 20–27 elégséges (2), 28–35 közepes (3), 36–43 jó (4), 44–50 jeles (5). 7. kérdés: írd ki minden tanuló nevét a jegyével.

Ellenőrzés: Anna 4, Bence 3, Csilla 5, Dani 1, Emma 3, Feri 4.

3.3. feladat ★☆☆ - Átlag felett

8. kérdés: sorold fel azokat, akik az átlagnál több pontot értek el.

Ellenőrzés: Kiss Anna, Toth Csilla, Kovacs Feri.

3.4. feladat ★☆☆ - Jegyek eloszlása

9. kérdés: melyik jegyből hány született? **Tipp:** használj egy hatelemű számláló vektort; a jegy legyen az index.

Ellenőrzés: elégtelen 1, elégséges 0, közepes 2, jó 2, jeles 1.

3.5. feladat ★★ - Rangsor és helyezés

10. kérdés: írd ki a tanulókat pontszám szerint csökkenő sorrendben. 11. kérdés: hányadik helyen végzett Varga Emma? (A 11. kérdéshez nem kell rendezni!)

Ellenőrzés: a rangsor élén Toth Csilla (47) áll, utána Kiss Anna és Kovacs Feri (42–42), majd Varga Emma (35), Nagy Bence (28), Szabo Dani (19). Varga Emma a 4. helyen végzett.

4. Hőmérséklet-napló

Amit gyakorolsz: vektor, a hat alapminta, szomszédos elemek összehasonlítása.

Egy meteorológiai állomás két héten át minden délben megmérte a hőmérsékletet. Az adatok (a napok sorszáma 1-től indul):

```
std::vector<int> hom = {3, 5, 4, -1, -2, 0, 2, 6, 7, 5, 8, 9, 4, 1};
```

4.1. feladat ★☆☆ - Átlag

Mennyi volt a két hét átlaghőmérséklete?

Ellenőrzés: 3.64286.

4.2. feladat ★☆☆ - Legmelegebb nap

Hányadik napon volt a legmelegebb, és hány fok volt?

Ellenőrzés: a 12. napon, 9 fok.

4.3. feladat ★☆☆ - Fagyos napok

Hány napon volt fagy (0 fok alatt)? Írd ki a napok sorszámát is.

Ellenőrzés: 2 napon: a 4. és az 5. napon.

4.4. feladat ★☆☆ - Volt-e?

Volt-e legalább 10 fokos nap? Volt-e olyan nap, amikor pontosan 0 fokot mértek?

Ellenőrzés: 10 fokos nap nem volt; 0 fokos volt.

4.5. feladat ★☆☆ - Átlag feletti napok

Írd ki azoknak a napoknak a sorszámát, amikor az átlagnál melegebb volt.

Ellenőrzés: 2 3 8 9 10 11 12 13 (nyolc nap).

4.6. feladat ★☆☆ - Legnagyobb melegedés

Melyik két egymást követő nap között nőtt a legtöbbet a hőmérséklet, és hány fokkal?

Ellenőrzés: a 7. és a 8. nap között, 4 fokkal.

4.7. feladat ★★ - Leghosszabb melegedő időszak

Hány napból áll a leghosszabb olyan időszak, amelyben minden nap melegebb volt az előzőnél? Melyik napon kezdődött?

Ellenőrzés: 5 napból áll, és az 5. napon kezdődött (-2, 0, 2, 6, 7).

4.8. feladat ★☆☆ - Függvényekkel

Szervezd át a programodat: írd `double atlag(std::vector<int> v)`, `int legmelegebb(std::vector<int> v)` (indexet ad vissza) és `int fagyosNapok(std::vector<int> v)` függvényt. A `main` csak meghívja őket.

Ellenőrzés: ugyanazok az eredmények, mint a 4.1–4.3. feladatban.

5. Szövegelemzés

Amit gyakorolsz: `string` bejárása, feldarabolás szavakra, karakterek vizsgálata, logikai függvény.

Egy mondatot kell elemezned. A mondat csupa kisbetűből és szóközökből áll:

```
std::string mondat = "a gorog kajak alatt egy level es egy radar volt a parton";
```

5.1. feladat ★☆☆ - Szavak száma

Darabold fel a mondatot szavakra egy vektorba, és írd ki, hány szóból áll.

Ellenőrzés: 12 szó.

5.2. feladat ★☆☆ - Leghosszabb szó

Melyik a leghosszabb szó, és hány betűs?

Ellenőrzés: `parton`, 6 betű.

5.3. feladat ★☆☆ - Magánhangzók

Hány magánhangzó (a, e, i, o, u) van a mondatban?

Ellenőrzés: 18.

5.4. feladat ★☆☆ - Palindrom szavak

Írd ki azokat a legalább hárombetűs szavakat, amelyek visszafelé olvasva is ugyanazok. Írj hozzá `bool palindrom(std::string s)` függvényt.

Ellenőrzés: `gorog`, `kajak`, `level`, `radar` (négy szó).

5.5. feladat ★☆☆ - Nagy kezdőbetűk

Írd ki a mondatot úgy, hogy minden szó nagybetűvel kezdődik.

Ellenőrzés: `A Gorog Kajak Alatt Egy Level Es Egy Radar Volt A Parton`

5.6. feladat ★★ - Különböző szavak

Hány különböző szó van a mondatban? Gyűjtsd ki őket egy új vektorba úgy, hogy minden szó csak egyszer szerepeljen.

Ellenőrzés: 10 különböző szó (az `a` és az `egy` kétszer fordul elő).

5.7. feladat ★★★ - Leggyakoribb betű

Melyik betű fordul elő a legtöbbször a mondatban, és hányszor?

Ellenőrzés: az `a` betű, 9-szer.

6. Belépőkódok

Amit gyakorolsz: karakterek vizsgálata, részsstring, szövegből szám, logikai függvények egymásra építése.

Egy rendezvény belépőkódjai így néznek ki: `KG-2041-7`. Egy kód akkor **érvényes**, ha két feltétel teljesül:

- A **formátuma** jó: pontosan 9 karakter; az első kettő nagybetű; a 3. és a 8. karakter kötőjel; a 4-7. karakter számjegy; az utolsó karakter is számjegy.
- Az **ellenőrző számjegye** jó: az utolsó számjegy megegyezik a középső négy számjegy összegének utolsó számjegyével. A példában $2 + 0 + 4 + 1 = 7$, tehát a kód érvényes.

A beérkezett kódok:

```
std::vector<std::string> kodok = {  
    "KG-2041-7", "AB-1234-0", "XY-9999-6", "kg-2041-7", "MN-4821-6",  
    "PQ-77A1-5", "RS-5005-0", "TU-123-6", "ZZ-0000-0", "EF-3172-4"  
};
```

6.1. feladat ★☆☆ - Számjegyek összege

Írj `int szamjegyOsszeg(std::string kod)` függvényt, amely a középső négy számjegy összegét adja vissza. (Ezt a függvényt csak jó formátumú kódra szabad meghívni.)

Ellenőrzés: `"KG-2041-7" → 7`, `"XY-9999-6" → 36`.

6.2. feladat ★★☆☆ - Formátum

Írj `bool formatumJo(std::string kod)` függvényt a fenti formátumszabály szerint.

Ellenőrzés: "KG-2041-7" → 1, "kg-2041-7" → 0, "PQ-77A1-5" → 0, "TU-123-6" → 0.

6.3. feladat ★★☆☆ - Érvényesség

Írj `bool ervenyos(std::string kod)` függvényt, amely az előző két függvényt használja.

Ellenőrzés: "KG-2041-7" → 1, "XY-9999-6" → 1, "MN-4821-6" → 0, "TU-123-6" → 0.

6.4. feladat ★★☆☆ - Összesítés

Hány érvényes kód érkezett? Írd ki az érvénytelen kódokat is, mindegyik mellé az okot: `hibas formatum` vagy `hibas ellenorzo szamjegy`.

Ellenőrzés: 5 érvényes kód. Hibás formátumú: `kg-2041-7`, `PQ-77A1-5`, `TU-123-6`. Hibás ellenőrző számjegyű: `MN-4821-6`, `EF-3172-4`.

6.5. feladat ★★☆☆ - Javítás

A hibás ellenőrző számjegyű kódokat javítsd ki: írd ki a helyes alakjukat.

Ellenőrzés: `MN-4821-5` és `EF-3172-3`.

6.6. feladat ★★☆☆ - Legnagyobb sorszám

Az érvényes kódok közül melyikben a legnagyobb a középső négyjegyű szám?

Ellenőrzés: `XY-9999-6`.

7. Parkolóház

Amit gyakorolsz: struct tagfüggvényekkel, sorok feldolgozása, számolás időpontokkal.

Egy parkolóház egy napi forgalma soronként érkezik, `rendszam;erkezes;tavozas` formában:

```
ABC-123;07:45;09:10
KLM-456;08:05;08:17
PRS-789;08:30;16:45
DEF-234;09:00;12:00
GHI-567;11:20;13:05
JKL-890;11:55;12:25
MNO-321;13:10;17:40
TUV-654;15:30;15:44
```

7.1. feladat ★☆☆☆ - Időpontból perc

Írj `int percben(std::string ido)` függvényt, amely egy `ora:perc` formájú időpontról megadja, hány perc telt el éjfél óta.

Ellenőrzés: "07:45" → 465, "12:00" → 720, "00:05" → 5.

7.2. feladat ★★☆☆ - Beolvasás

Írj `Parkolas` structot három mezővel: `rendszam`, valamint `be` és `ki` (az érkezés és a távozás percben).

Legyen `int idotartam()` tagfüggvénye. Írj `Parkolas feldolgoz(std::string sor)` függvényt, és töltsd be a nyolc sort egy vektorba.

Ellenőrzés: 8 parkolás; az első autó időtartama 85 perc.

7.3. feladat ★☆☆☆ - Leghosszabb parkolás

Melyik autó parkolt a leghosszabb ideig, és mennyit? Mennyi az átlagos parkolási idő?

Ellenőrzés: `PRS-789`, 495 perc (8 óra 15 perc). Az átlag 148.875 perc.

7.4. feladat ★★☆☆ - Parkolási díj

Írj a structba `int dij()` tagfüggvényt. A szabály: a legfeljebb 15 perces parkolás ingyenes; egyébként minden megkezdett óra 400 Ft.

Ellenőrzés: 12 perc → 0 Ft, 30 perc → 400 Ft, 85 perc → 800 Ft, 180 perc → 1200 Ft, 495 perc → 3600 Ft.

7.5. feladat ★★☆☆ - Bevétel

Mennyi volt a parkolóház napi bevétele? Hány autónak nem kellett fizetnie?

Ellenőrzés: 8800 Ft; 2 autó parkolt ingyen.

7.6. feladat ★★☆☆ - Délben

Hány autó állt a parkolóházban pontban 12:00-kor? Egy autó akkor áll bent, ha legkésőbb 12:00-kor érkezett, és 12:00 után távozott. Írd ki a rendszámokat is.

Ellenőrzés: 3 autó: `PRS-789`, `GHI-567`, `JKL-890`.

7.7. feladat ★★☆☆ - Fizetési lista

Írd ki a rendszámokat a fizetendő díjjal együtt, a legnagyobb díjtól a legkisebbig.

Ellenőrzés: az első három sor: `PRS-789 3600`, `MNO-321 2000`, `DEF-234 1200`. Az utolsó két sorban 0 Ft áll.

8. Osztálynapló

Amit gyakorolsz: struct, amelyben vektor van; kétszintű feldarabolás; minták egymásba ágyazva.

Egy osztály naplójának sorai `nev`; `jegyek` formájúak. A jegyeket szóköz választja el, és nem mindenkinek ugyanannyi jegye van:

```
Kiss Anna;5 4 5 3 5
Nagy Bence;3 2 4 3
Toth Csilla;5 5 4 5 5 5
Szabo Dani;2 1 3 2
Varga Emma;4 4 5
Kovacs Feri;3 4 1 5 4
```

8.1. feladat ★★☆☆ - Beolvasás

Írj `Tanulo` structot két mezővel: `nev` és `jegyek` (egész számok vektora). Írj `Tanulo feldolgoz(std::string sor)` függvényt, és töltsd be a hat sort egy vektorba.

Ellenőrzés: 6 tanuló; Kiss Annának 5 jegye, Varga Emmának 3 jegye van.

8.2. feladat ★☆☆☆ - Átlagok

Írj a structba `double atlag()` tagfüggvényt, és írd ki minden tanuló nevét az átlagával.

Ellenőrzés: Anna 4.4, Bence 3, Csilla 4.83333, Dani 2, Emma 4.33333, Feri 3.4.

8.3. feladat ★☆☆☆ - A legjobb tanuló

Kinek a legjobb az átlaga?

Ellenőrzés: Toth Csilla.

8.4. feladat ★★☆☆ - Bukásra áll?

Írj a structba `bool vanEgyes()` tagfüggvényt, és sorold fel azokat, akiknek van egyes jegye.

Ellenőrzés: Szabo Dani és Kovacs Feri.

8.5. feladat ★★☆☆ - Ötösök

Hány ötös született összesen az osztályban? Kinek van a legtöbb ötöse?

Ellenőrzés: 10 ötös; a legtöbb Toth Csillának van (5 darab).

8.6. feladat ★★★ - Osztályátlag

Mennyi az osztályban kiosztott összes jegy átlaga?

Ellenőrzés: 3.74074 (27 jegy, az összegük 101). Számold ki a tanulói átlagok átlagát is: az 3.66 körül van. Miért nem egyezik a kettő? Melyik a „valódi” osztályátlag?

8.7. feladat ★★☆☆ - Osztályátlag felett

Sorold fel azokat, akiknek az átlaga jobb az osztályátlagnál.

Ellenőrzés: Kiss Anna, Toth Csilla, Varga Emma.

8.8. feladat ★★★ - Rangsor

Írd ki a tanulókat átlag szerint csökkenő sorrendben.

Ellenőrzés: Toth Csilla, Kiss Anna, Varga Emma, Kovacs Feri, Nagy Bence, Szabo Dani.

9. Bajnokság

Amit gyakorolsz: két struct együtt, különböző értékek kigyűjtése, adatok összesítése csapatonként, rendezés két szempont szerint. Ez a feladatlap legnehezebb fejezete.

Egy négycsapatos bajnokság mérkőzései `hazai;vendeg;hazai_golok;vendeg_golok` formában érkeznek:

```
Sasok;Farkasok;3;1
Medvek;Rokak;2;2
Sasok;Medvek;0;1
Farkasok;Rokak;4;2
Rokak;Sasok;1;3
Farkasok;Medvek;2;2
```

9.1. feladat ★★☆☆ - Beolvasás

Írj `Meccs` structot négy mezővel (`hazai`, `vendeg`, `hazaiGol`, `vendegGol`) és egy `int golok()` tagfüggvénnyel, amely a mérkőzés összes gólját adja vissza. Töltsd be a hat sort.

Ellenőrzés: 6 mérkőzés; az elsőn 4 gól esett.

9.2. feladat ★☆☆☆ - Gólok

Hány gól esett összesen, és mennyi a mérkőzésenkénti gólátlag? Hány mérkőzés végződött döntetlenre?

Ellenőrzés: 23 gól, az átlag 3.83333; 2 döntetlen.

9.3. feladat ★☆☆☆ - A leggólgazdagabb mérkőzés

Melyik mérkőzésen esett a legtöbb gól? Írd ki így: `Farkasok - Rokak 4:2`

Ellenőrzés: a fenti sor (6 gól).

9.4. feladat ★★★ - Csapatok

Gyűjtsd ki egy vektorba a bajnokságban szereplő csapatok nevét úgy, hogy mindegyik csak egyszer szerepeljen. Figyelj: egy csapat hazaként és vendégként is előfordul.

Ellenőrzés: 4 csapat: Sasok, Farkasok, Medvek, Rokak.

9.5. feladat ★★ - Tabella

Írj `Csapat` structot: `nev`, `gyozelem`, `dontetlen`, `vereseg`, `lottGol`, `kapottGol`. Legyen `int pont()` tagfüggvénye (győzelem 3 pont, döntetlen 1 pont). Készíts minden csapatnak egy `Csapat` elemet csupa nullával, majd menj végig a mérkőzéseken: minden mérkőzés **két** csapat adatait módosítja.

Ellenőrzés:

Csapat	Győzelem	Döntetlen	Vereség	Lőtt gól	Kapott gól	Pont
Sasok	2	0	1	6	3	6
Farkasok	1	1	1	7	7	4
Medvek	1	2	0	5	4	5
Rokak	0	1	2	5	9	1

9.6. feladat ★★ - Végeredmény

Írd ki a bajnokság végeredményét: a csapatokat pontszám szerint csökkenő sorrendben. Azonos pontszámnál a jobb gólkülönbség (lőtt mínusz kapott gól) döntson.

Ellenőrzés: 1. Sasok (6 pont), 2. Medvek (5), 3. Farkasok (4), 4. Rokak (1).

9.7. feladat ★★ - Veretlenül

Volt-e olyan csapat, amely egyetlen mérkőzést sem veszített el? Ha igen, melyik?

Ellenőrzés: igen, a Medvek.

Segítség a feladatokhoz

Csak akkor olvasd el egy feladat segítségét, ha már legalább öt percig próbálkoztál vele. A segítség nem megoldás: azt mondja meg, merre indulj.

2. fejezet

- **2.1.** (1) megszámolás; (2) összegzés; (3) maximumkiválasztás; (4) eldöntés; (5) keresés; (6) kiválogatás; (7) eldöntés; (8) rendezés; (9) megszámolás: hányak van több pontja, mint Emmának? A helyezés ennél eggyel nagyobb.

3. fejezet

- **3.1.** Minimumkiválasztás: a 3. kérdés mintája, fordított relációjellel.
- **3.2.** A tagfüggvényben `if - else if` lánc, minden ágon `return`.
- **3.3.** Az átlag már megvan a 2. kérdésből. Kiválogatás (vagy egyszerűen kiírás a ciklusban).
- **3.4.** `std::vector<int> darab(6);` - a ciklusban `darab[v[i].jegy()]++`;
- **3.5.** Rendezés összehasonlító függvénnyel. A helyezéshez számold meg, hányak van több pontja Emmának.

4. fejezet

- **4.1.** Összegzés; az összeg legyen `double`.
- **4.2.** Maximumkiválasztás indexszel; a nap sorszáma az index + 1.
- **4.4.** Két külön eldöntés, két `bool` változóval.
- **4.5.** Előbb az átlag kell, utána egy második ciklus a kiíráshoz.
- **4.6.** A ciklusban a `hom[i + 1] - hom[i]` különbségek közül keresd a legnagyobbat. Meddig mehet az `i`?
- **4.7.** Vezess egy `hossz` változót: ha a mai nap melegebb az előzőnél, növel; ha nem, állítsd vissza 1-re. Közben jegyezd meg az eddigi legnagyobb hosszt, és azt, hogy melyik indexnél indult a szakasz.

5. fejezet

- **5.1.** `std::stringstream` és `while (ss >> szo)`, a szavakat `push_back`-kel gyűjtsd.
- **5.2.** Maximumkiválasztás a szavak `length()` értéke szerint.
- **5.3.** Járd be a mondatot karakterenként; érdemes `bool maganhangzo(char c)` függvényt írni.
- **5.4.** Fordítsd meg a szót egy új stringben, és hasonlítsd össze az eredetivel.
- **5.5.** Minden szó első karakterét alakítsd át a `std::toupper` függvénnyel, a kiírásnál szóközt tegyél a szavak közé.
- **5.6.** Egy szót csak akkor tegyél az új vektorba, ha még nincs benne. Írj `bool vanBenne(std::vector<std::string> v, std::string szo)` függvényt.
- **5.7.** `std::vector<int> darab(26);` - minden betűnél `darab[c - 'a']++`; (a szóközöket hagyd ki). Utána maximumkiválasztás a számlálók között; a betűt a `char('a' + index)` adja vissza.

6. fejezet

- **6.1.** A számjegyek a 3., 4., 5. és 6. indexen állnak. Egy számjegy karakter értéke: `kod[i] - '0'`.
- **6.2.** Először a hosszt ellenőrizd, és ha nem 9, azonnal `return false`; - különben túlindexelsz. Utána karakterenként: `std::isupper`, `std::isdigit`, illetve `== '-'`.
- **6.3.** `return formatumJo(kod) && ...`; - az ellenőrző számjegy értéke `kod[8] - '0'`, az elvárt érték `samjegyOsszeg(kod) % 10`.
- **6.4.** Előbb a formátumot vizsgáld; az ellenőrző számjegyet csak akkor, ha a formátum jó.
- **6.5.** `kod.substr(0, 8)` a kód eleje; ehhez fűzd hozzá a helyes számjegyet a `std::to_string` segítségével.
- **6.6.** `std::stoi(kod.substr(3, 4))` a négyjegyű szám. Maximumkiválasztás, de csak az érvényes kódok között.

7. fejezet

- **7.1.** Az óra a `substr(0, 2)`, a perc a `substr(3, 2)`; mindkettőt `std::stoi` alakítja számmá.
- **7.2.** A `feldolgoz` függvényben három `std::getline(ss, ..., ';')`; a két időpontot a `percben` függvénnyel alakítsd át.
- **7.4.** A megkezdett órák száma egész osztással: `(perc + 59) / 60`.
- **7.5.** Összegzés a `dij()` értékekre, megszámlálás a `dij() == 0` feltétellel.
- **7.6.** Megszámlálás összetett feltétellel: `be <= 720 && ki > 720`.
- **7.7.** Rendezés összehasonlító függvénnyel: `return a.dij() > b.dij();`

8. fejezet

- **8.1.** Két lépésben darabolj. Első lépés: `std::getline(ss, t.nev, ';')`; majd `std::getline(ss, maradék);`. Második lépés: egy új `std::stringstream` a `maradék` szövegre, és `while (ss2 >> jegy)`.
- **8.4.** Eldöntés a `jegyek` vektoron, a tagfüggvényen belül.
- **8.5.** Egymásba ágyazott ciklus: kívül a tanulók, belül a jegyeik. Érdemes `int hany0tos()` tagfüggvényt írni.
- **8.6.** Két számlálót vezess: az összes jegy összegét és az összes jegy darabszámát.
- **8.8.** Összehasonlító függvény: `return a.atlag() > b.atlag();`

9. fejezet

- **9.1.** A `feldolgoz` függvényben négy `std::getline(ss, ..., ';')`, a két gólszámot `std::stoi` alakítja át.
- **9.4.** Az 5.6. feladat ötlete: egy nevet csak akkor tegyél a vektorba, ha még nincs benne. Minden mérkőzésnél a hazai és a vendég csapatot is vizsgáld meg.
- **9.5.** Írj `int keres(std::vector<Csapat> v, std::string nev)` függvényt, amely megadja a csapat indexét. Minden mérkőzésnél keresd meg a két csapat indexét (`h` és `g`), aztán módosítsd a `csapatok[h]` és a `csapatok[g]` mezőit. Három eset van: hazai győzelem, döntetlen, vendég győzelem.
- **9.6.** Az összehasonlító függvényben: ha a két pontszám különbözik, az dönt; ha egyenlő, a gólkülönbség.
- **9.7.** Keresés vagy eldöntés a `veresege == 0` feltétellel.